

BAB II

PEMILIHAN SOLUSI

2.1 Alternatif Solusi

2.1.1 *K-Nearest Neighbors*

Solusi pertama adalah menggunakan *K-Nearest Neighbors* (KNN) sebagai metode *supervised learning* yang sederhana dalam *machine learning*. Metode ini bekerja dengan mengklasifikasikan data baru berdasarkan kesamaan dengan data yang telah dilatih sebelumnya, di mana kesamaan tersebut dihitung dari jarak antara data menggunakan nilai k sebagai jumlah tetangga terdekat (Kaur, 2024).

Dalam menentukan kesamaan berdasarkan jumlah tetangga terdekat, KNN mengukur jarak antara data latih dan uji menggunakan jenis perhitungan jarak (Tace, 2022). Salah satu metrik jarak yang digunakan adalah persamaan *manhattan* yang didefinisikan persamaan (2.1).

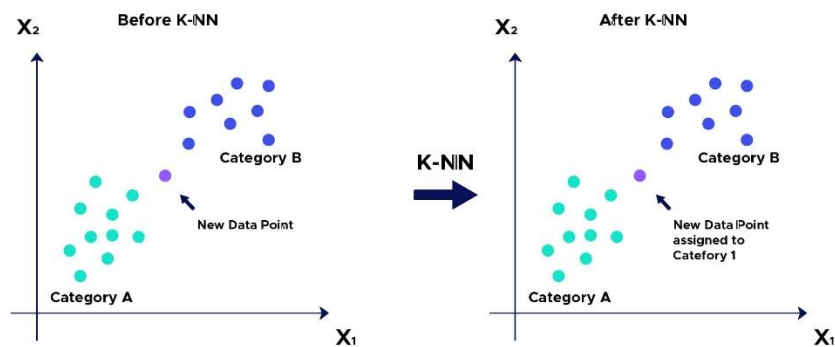
$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (2.1)$$

di mana $d(x, y)$ adalah jarak antara vektor x (data pelatihan) dan y (data pengujian), n adalah jumlah atribut, dan i adalah indeks atribut.

Kelebihan KNN terletak pada kesederhanaan dalam menentukan parameter jumlah tetangga terdekat (k) dan metode pengukuran jarak. Metode ini mampu mengelola sampel data yang besar, meski dapat berdampak pada waktu pemrosesan (Abaye, 2020). Namun, pemilihan nilai k harus dipertimbangkan dengan teliti. Nilai k yang kecil sangat rentan terhadap *noise* dan menyebabkan *overfitting*.

Sementara nilai k yang besar dapat mengurangi pengaruh dari *noise*, meskipun meningkatkan kompleksitas komputasi (Cholil, 2021).

Kelemahan utama dari KNN ketergantungannya pada kualitas dan ukuran dataset. Metode ini sangat sensitif *outlier* dan data yang hilang (*missing*), serta kurang cocok untuk data yang berdimensi tinggi (Rahmadini, 2023). KNN memerlukan komputasi yang intensif dan waktu pemrosesan akan meningkat seiring dengan penambahan ukuran dataset (Abaye, 2020).



Gambar 2.1 *K-Nearest Neighbors*

Gambar 2.1 menunjukkan metode *K-Nearest Neighbors* (KNN) terdiri dari beberapa tahapan dalam proses klasifikasi, yaitu:

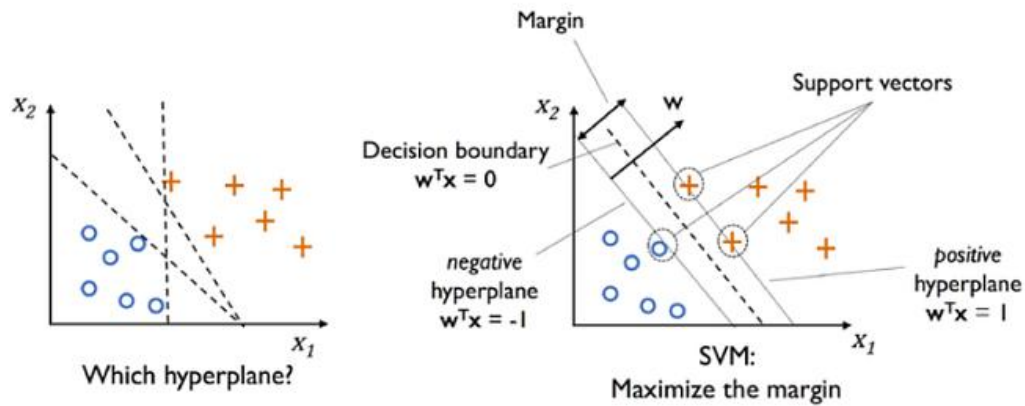
1. Memilih parameter jumlah tetangga terdekat (k).
2. Menghitung jarak setiap data uji dengan data latih menggunakan persamaan *manhattan distance*.
3. Mengurutkan hasil perhitungan jarak *manhattan* dimulai dari yang terkecil hingga yang terbesar.
4. Mengklasifikasikan data uji berdasarkan kelas mayoritas yang paling sering muncul dari (k) tetangga terdekat.

Pada solusi pertama, perancangan sistem irigasi cerdas menggunakan *K-Nearest Neighbors* (KNN), parameter yang digunakan adalah $k = 5$ dan jenis jarak yang digunakan adalah *manhattan*. Dataset ini dibagi menjadi 80% data latih dan 20% data uji, menghasilkan akurasi sebesar 99,8% (Kaur, 2024).

2.1.2 *Support Vector Machine*

Solusi kedua adalah menggunakan *Support Vector Machine* (SVM) sebagai metode *supervised learning* dalam *machine learning*, yang mampu menyelesaikan masalah klasifikasi dan regresi. SVM bekerja dengan mencari *hyperplane* terbaik yang memisahkan dua kelas dengan memaksimalkan margin. Metode ini mampu memisahkan data secara linier maupun non-linier. SVM memiliki kelebihan dalam menghasilkan generalisasi yang tinggi, tahan terhadap noise dan outlier, serta mampu mengelola data yang tidak seimbang. Tidak hanya itu, SVM dapat menangani data berdimensi tinggi dengan jumlah sampel terbatas (Abaye, 2020).

Kelemahan dari SVM adalah tidak cocok untuk himpunan data dengan jumlah sampel dan dimensi yang besar. Kinerja SVM dipengaruhi dalam pemilihan *kernel* dan nilai regulasi C , sehingga akurasi, kebutuhan komputasi dan waktu pemrosesan bervariasi (Abaye, 2020).



Gambar 2.2 Support Vector Machine

Gambar 2.2 menunjukkan proses kerja dalam menentukan *hyperplane* dan memaksimalkan *margin*. SVM terdiri dari tiga komponen utama, yaitu *hyperplane* sebagai bidang pemisah terbaik antara dua kelas, *margin* yang merupakan jarak *hyperplane* dan jarak terdekat dari masing-masing kelas, serta *support vector* yang merupakan data yang paling dekat dengan *hyperplane* (Junus, 2023).

Proses pelatihan SVM dimulai dengan himpunan data pelatihan $(x_1, y_1), (x_2, y_2), \dots, (x_i, y_i)$, di mana $x_i \in R^n$ mewakili vektor fitur, dan $y_i \in R^n$ menunjukkan label kelas. Kedua kelas dapat memisahkan dengan sempurna oleh *hyperplane*, yang didefinisikan dengan persamaan (2.2).

$$w^T \cdot x + b = 0 \quad (2.2)$$

di mana w adalah vektor bobot yang terkait dengan masing-masing fitur data, x adalah vektor fitur data, dan b adalah nilai bias.

Tujuan dari SVM adalah menemukan *hyperplane* yang memaksimalkan *margin*, yaitu jarak antara dua kelas terhadap *hyperplane*. *Margin* tersebut dihitung sebagai $\frac{2}{\|w\|}$, di mana $\|w\|$ merupakan norma dari vektor bobot w .

Untuk menemukan *hyperplane* terbaik, dilakukan proses optimasi melalui *Quadratic Programming* dengan meminimalkan fungsi pada persamaan (2.3).

$$\min \frac{1}{2} \|w\|^2 \quad (2.3)$$

dengan syarat:

$$y_i(w \cdot x_i + b) \geq 1, \quad i = 1, 2, 3, \dots, n \quad (2.5)$$

Optimalisasi dapat diselesaikan dengan menerapkan metode *lagrange multiplier* yang ditunjukkan pada persamaan (2.6).

$$L_p(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i(w \cdot x_i + b) - 1] \quad (2.6)$$

di mana $\alpha_i \geq 0$ adalah nilai *lagrange multiplier*.

Untuk memperoleh nilai optimal dari w dan b , dilakukan penurunan fungsi L_p terhadap w dan b , kemudian diubah menjadi nilai nol, sehingga menghasilkan syarat optimal yang ditunjukkan pada persamaan (2.7) dan persamaan (2.8).

$$\frac{\delta L}{\delta w} = 0 \rightarrow \sum_{i=1}^n \alpha_i x_i y_i = w \quad (2.7)$$

$$\frac{\delta L}{\delta b} = 0 \rightarrow \sum_{i=1}^n \alpha_i y_i = 0 \quad (2.8)$$

Persamaan *lagrange multiplier* diubah dengan substitusi turunan dari persamaan (2.7) dan persamaan (2.8), sehingga membentuk dualitas *lagrange*. Kemudian memaksimalkan dengan persamaan (2.9).

$$\max_{\alpha} L_d = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \quad (2.9)$$

dengan syarat: $\alpha_i \geq 0$; $i = 1, 2, 3, \dots, n$; $\sum_{i=1}^n \alpha_i y_i = 0$.

Hasil dari persamaan (2.9) adalah $\alpha_i > 0$, yang disebut sebagai *support vectors* untuk menentukan posisi *hyperplane*. Untuk memprediksi kelas dari data baru yang dimasukkan, dengan menggunakan persamaan (2.10).

$$f(x) = \left(\sum_{i,j=1}^n \alpha_i y_i x_i x_j \right) + b \quad (2.10)$$

di mana x_i adalah *support vector*, dan x_j sebagai vektor data baru yang akan diprediksi.

Untuk kasus kedua kelas yang tidak dapat dipisahkan secara sempurna, SVM menawarkan *kernel trick* untuk memisahkan data non-linier dengan mentransformasikan data ke ruang fitur berdimensi lebih tinggi (Kaur, 2024). Namun, dalam penelitian tersebut, menerapkan *kernel linear* yang pengukuran produk titik dari vektor data di ruang aslinya, tanpa melakukan transformasi (Rabbani, 2023). Umumnya *kernel linear* mampu tahan *overfitting* dan proses pelatihan lebih cepat dibandingkan kernel non *linear* (Abaye, 2020). Persamaan kernel linear dapat didefinisikan pada persamaan (2.11).

$$k(x_i \cdot x_j) = x_i \cdot x_j \quad (2.11)$$

Selain menerapkan *kernel trick*, SVM juga mampu menyelesaikan kasus tersebut dengan menerapkan *soft margin* dengan menambahkan variabel *slack* $\xi > 0$ yang berfungsi sebagai pengukur kesalahan dalam klasifikasi. Variabel ini dimasukkan ke dalam bentuk *Quadratic Programming* pada persamaan (2.12).

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (2.12)$$

dengan syarat:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad i = 1, 2, 3, \dots, n \quad (2.13)$$

Masalah optimasi dengan *lagrange multiplier* pada persamaan (2.6) dimodifikasi menjadi persamaan berikut:

$$L_p(w, b, \alpha) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i [y_i(w \cdot x_i + b) - 1 + \xi_i] - \sum_{i=1}^n \mu_i \xi_i \quad (2.14)$$

Persamaan dari maksimum dualitas *lagrange multiplier* dilakukan dengan memodifikasi menjadi persamaan (2.15).

$$\max_{\alpha} L_p = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \quad (2.15)$$

dengan syarat: $0 \leq \alpha_i \leq C$; $i = 1, 2, 3, \dots, n$; $\sum_{i=1}^n \alpha_i y_i = 0$.

Parameter C yang digunakan untuk mengatur kesalahan klasifikasi yang diperbolehkan pada data latih. Nilai yang besar akan memperketat dan mengurangi toleransi kesalahan dengan memperkecil margin, namun berisiko *overfitting*. Sebaliknya, nilai yang kecil akan memberikan toleransi kesalahan dengan melebarkan margin, namun berisiko *underfitting* (Aryawan, 2023).

Pada solusi kedua, perancangan sistem irigasi menggunakan *Support Vector Machine* (SVM), *kernel* yang digunakan adalah *linear*. Dataset ini dibagi menjadi 80% data latih dan 20% data uji, menghasilkan akurasi sebesar 99,3% (Kaur, 2024).

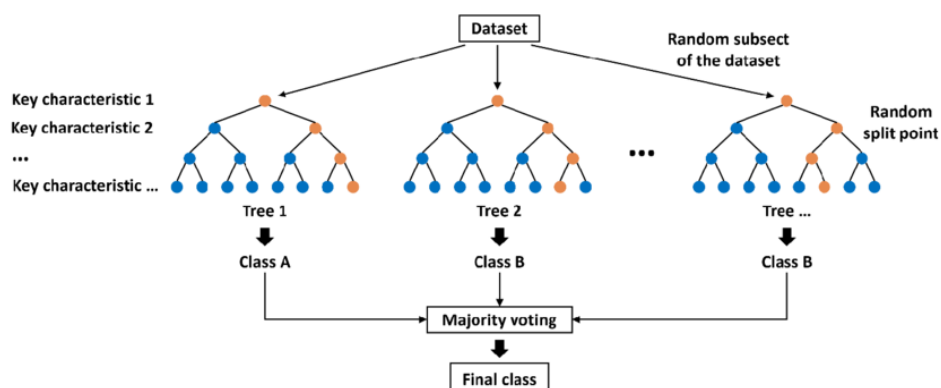
2.1.3 *Random Forest*

Solusi ketiga menggunakan *Random Forest* (RF) adalah metode ensemble learning yang populer untuk klasifikasi maupun regresi. Metode ini bekerja dengan menggabungkan sejumlah *Decision Tree* yang terpisah, di mana setiap pohon menggunakan subset data yang dipilih secara acak melalui teknik bagging dan *random feature selection* (Abaye, 2020). *Random Forest* dikembangkan dari *Classification* dan *Regression Tree* (CART), yang merupakan teknik klasifikasi yang menganalisis data menggunakan *Decision Tree* (Lestari, 2024).

Setiap *Decision Tree* terdiri dari beberapa jenis simpul, yaitu simpul akar yang terletak di puncak dalam pohon. Simpul internal sebagai simpul percabangan yang memiliki satu *input* dan dua *output*, dan simpul daun. Simpul terakhir yang dikenal sebagai simpul daun yang hanya memiliki satu masukan tanpa menghasilkan keluaran tambahan (Rakhmat, 2023).

Random Forest memiliki kemampuan generalisasi yang tinggi dengan membangun pohon yang beragam, sehingga mengurangi *overfitting*. Metode ini mampu memproses data dengan jumlah sampel dan dimensi yang besar, serta fitur kategoris. Keunggulan metode ini tahan terhadap data dengan *outlier*, *noise*, dan ketidakseimbangan (Abaye, 2020). Tidak hanya itu, *Random Forest* mampu menangani missing data (Zhu, 2020).

Kelemahan dari *Random Forest* terletak pada kebutuhan komputasi dan waktu pemrosesan yang dapat meningkat seiring bertambahnya jumlah dan kedalaman pohon, terutama dataset yang besar. Metode ini juga memerlukan spesifikasi prosesor tinggi untuk pemrosesan paralel. Oleh karena itu, pengaturan *hyperparameter* untuk membatasi jumlah dan kedalaman maksimum pohon yang dibangun sangat penting untuk mengurangi kebutuhan komputasi (Zhu, 2020).



Gambar 2.3 Random Forest

Berdasarkan Gambar 2.3, *Random Forest* membangun sejumlah *Decision Tree* yang secara terpisah melalui teknik *bagging* (*bootstrap aggregating*), di mana sampel acak dari data pelatihan yang dipilih dengan penggantian (*replacement*). Setiap pohon dibangun tanpa pemangkasan dan menggunakan *random feature selection*. Proses ini diulangi hingga jumlah pohon yang ditentukan (Lestari, 2024).

Random feature selection dilakukan untuk memilih sejumlah fitur secara acak yang digunakan pada setiap simpul saat membangun pohon. Dalam pengaturan ini, nilai yang lebih rendah dapat meningkatkan variasi dan mengurangi korelasi antar pohon, yang dapat memperkuat stabilitas saat penggabungan pohon (Yulianto, 2024). Untuk klasifikasi, nilai ini sering diatur dengan $\sqrt{p}$, di mana p adalah jumlah fitur dalam dataset (Junus, 2023).

Proses pembentukan *Decision Tree* dalam *Random Forest* dimulai dengan menentukan simpul akar. Selanjutnya, dilakukan pemisahan pada simpul untuk menentukan simpul terminal dan simpul daun berdasarkan kriteria terbaik dengan menghitung menggunakan *Gini Index* (Yulianti, 2023). Perhitungan *Gini Index* dan *Gini Split* dilakukan menggunakan persamaan (2.16) dan persamaan (2.17).

$$Gini\ Index\ (D) = 1 - \sum_{i=1}^n P_i^2 \quad (2.16)$$

di mana P_i adalah probabilitas dari kelas ke- i dalam simpul D , dan n adalah jumlah total kelas.

$$Gini_{split}\ (D) = \frac{N_1}{N} Gini\ Index(D_1) + \frac{N_2}{N} Gini\ Index(D_2) \quad (2.17)$$

di mana N_1 dan N_2 adalah jumlah sampel dalam subset setelah pemisahan, D_1 dan D_2 adalah simpul hasil pemisahan, serta N adalah jumlah total sampel pada simpul.

Pada solusi ketiga, perancangan sistem irigasi menggunakan *Random Forest* (RF), dengan membangun jumlah 100 pohon dan kriteria *Gini Index*. Dataset tersebut akan dibagi menjadi 80% untuk data latih dan 20% untuk data uji. *Random Forest* menghasilkan akurasi sebesar 99,98%, *precision* 99,96%, *recall* 99,99%, dan *f1-score* 99,97% (IORLIAM, 2022).

2.2 Analisis Pemilihan Solusi

Tabel 2.2 Analisis Pemilihan Solusi

Solusi	Karakteristik Pendekatan Berdasarkan Aspek Analisis	Solusi Terpilih
Pendekatan-1 Preprocessing: - <i>train_test_split</i> (80 : 20) - <i>StandarScaler</i> Classifier: <i>K-Nearest Neighbors</i> <i>(k: 5, metric: manhattan)</i>	Aspek Ekonomis Selama proses pelatihan KNN, kebutuhan komputasi bisa meningkat seiring bertambahnya ukuran dataset (Abaye, 2020). Aspek Manufakturabilitas Waktu pemrosesan untuk melatih KNN menjadi lama seiring bertambahnya ukuran dataset (Abaye, 2020). Aspek Sustainabilitas Kemampuan pada KNN menghasilkan akurasi mencapai 99,3% (Kaur, 2024).	-
Pendekatan-2 Preprocessing: - <i>train_test_split</i> (80 : 20) - <i>StandarScaler</i>	Aspek Ekonomis Pemilihan <i>kernel</i> dan ukuran dataset seperti jumlah sampel yang besar dapat meningkatkan kebutuhan komputasi selama proses pelatihan (Abaye, 2020). Namun, SVM ramah dalam pengguna memori	-

Solusi	Karakteristik Pendekatan Berdasarkan Aspek Analisis	Solusi Terpilih
Classifier: <i>Support Vector Machine</i> (<i>kernel: linear</i>)	dengan hanya membangun batas keputusan menggunakan subset data pelatihan yang disebut <i>support vector</i> (Palaniappan, 2024). Aspek Manufakturabilitas Pemilihan kernel dan parameter <i>C</i> sangat memengaruhi proses pelatihan SVM. Umumnya, <i>kernel linear</i> jauh lebih cepat dibandingkan <i>non-linear</i>. Sementara itu, ukuran dataset yang sangat besar membuat proses pelatihan menjadi sangat lambat (Abaye, 2020). Aspek Sustainabilitas Kemampuan pada SVM menghasilkan akurasi sebesar 99,3% (Kaur, 2024).	
Pendekatan-3 Preprocessing: <i>train_test_split</i> (80 : 20) Classifier: <i>Random Forest</i> (<i>n_estimators</i>: 100 , <i>criterion</i>: 'gini', <i>max_features</i>: 'sqrt', <i>bootstrap</i>: True)	Aspek Ekonomis Selama melatih RF membutuhkan komputasi yang tinggi maupun rendah tergantung ukuran dataset, serta jumlah pohon dan kedalaman pohon yang dibangun. Namun kebutuhan komputasi tersebut dapat diminimalkan dengan membatasi kedalaman pohon yang dibangun (Zhu, 2020). Kedalaman pohon terendah yang mampu menghasilkan kinerja klasifikasi yang baik dan tetap konsisten akan dipilih untuk meminimalkan kebutuhan komputasi. Aspek Manufakturabilitas	✓

Solusi	Karakteristik Pendekatan Berdasarkan Aspek Analisis	Solusi Terpilih
	Selain ukuran dataset, <i>hyperparameter</i> seperti jumlah pohon dan kedalaman pohon dapat memengaruhi waktu pelatihan. Untuk mempercepat proses tersebut dapat dilakukan dengan menerapkan pemrosesan paralel dan mengatur <i>hyperparameter</i> dengan membatasi kedalaman pohon, selain jumlah pohon yang dibangun (Zhu, 2020). Aspek Sustainbilitas Kemampuan pada RF menghasilkan akurasi mencapai 99,98% (IORLIAM, 2022).	